PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Charles Santos de Souza Meira
Paulo Dimas da Silva Júnior

# Trabalho Prático 2: Banco de Registradores, Registrador de Status, Pilha e Contador de Programa em VHDL

Belo Horizonte

2026

Charles Santos de Souza Meira
Paulo Dimas da Silva Júnior

# Trabalho Prático 2: Banco de Registradores, Registrador de Status, Pilha e Contador de Programa em VHDL

Relatório do Trabalho Prático 2 apresentado à disciplina Sistemas Reconfiguráveis do curso de Engenharia de Computação da Pontifícia Universidade Católica de Minas Gerais, referente ao projeto, à simulação e à validação de quatro blocos sequenciais descritos em VHDL: banco de registradores, registrador de status, pilha e contador de programa.

Pontifícia Universidade Católica de Minas Gerais

Professor: Francisco Garcia

Belo Horizonte

2026

# RESUMO

Este relatório descreve o projeto de quatro blocos sequenciais descritos em linguagem VHDL e sintetizados no Quartus II 9.1sp2: um banco com quatro registradores de 8 bits e duas saídas de leitura independentes (*Reg_bank*); um registrador de status com cinco registradores de 1 bit habilitados individualmente (*Status_reg*); uma pilha com oito registradores de 11 bits e operações de *push* e *pop* (*Stack*); e um contador de programa módulo 2048 com carga síncrona e *reset* assíncrono (*Prog_cnt*). Para cada bloco são apresentados a especificação, as decisões de projeto, o código comentado, o resultado da compilação e os casos de teste. A validação foi feita por simulação funcional, comparando os valores obtidos com os valores esperados calculados a partir da especificação.

**Palavras-chave**: VHDL. Código sequencial. Registradores. Pilha. Contador de programa. Quartus II.

# LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama funcional do *Reg_bank* . . . . . 11
Figura 2 – Simulação do *Reg_bank* . . . . . 13
Figura 3 – Diagrama funcional do *Status_reg* . . . . . 15
Figura 4 – Simulação do *Status_reg* . . . . . 17
Figura 5 – Diagrama funcional da *Stack* . . . . . 19
Figura 6 – Simulação da *Stack* . . . . . 22
Figura 7 – Diagrama funcional do *Prog_cnt* . . . . . 24
Figura 8 – Simulação do *Prog_cnt* . . . . . 27

# LISTA DE TABELAS


Tabela 1 – Entradas e saídas do *Reg_bank* . . . . . 9
Tabela 2 – Resumo da compilação do bloco *Reg_bank* no Quartus II 9.1sp2 . . . . . 12
Tabela 3 – Casos de teste do *Reg_bank* . . . . . 13
Tabela 4 – Correspondência entre os bits e os registradores do *Status_reg* . . . . . 14
Tabela 5 – Entradas e saídas do *Status_reg* . . . . . 14
Tabela 6 – Resumo da compilação do bloco *Status_reg* no Quartus II 9.1sp2 . . . . . 16
Tabela 7 – Casos de teste do *Status_reg* . . . . . 17
Tabela 8 – Entradas e saídas da *Stack* . . . . . 18
Tabela 9 – Resumo da compilação do bloco *Stack* no Quartus II 9.1sp2 . . . . . 20
Tabela 10 – Casos de teste da *Stack* . . . . . 21
Tabela 11 – Funções selecionadas por `pc_ctrl` . . . . . 23
Tabela 12 – Entradas e saídas do *Prog_cnt* . . . . . 23
Tabela 13 – Resumo da compilação do bloco *Prog_cnt* no Quartus II 9.1sp2 . . . . . 26
Tabela 14 – Casos de teste do *Prog_cnt* . . . . . 27

# LISTA DE CÓDIGOS


2.1 Entidade do *Reg_bank* . . . . . 10
2.2 Registradores e escrita do *Reg_bank* . . . . . 10
2.3 Leitura assíncrona do *Reg_bank* . . . . . 11
3.1 Entidade do *Status_reg* . . . . . 15
3.2 Arquitetura do *Status_reg* . . . . . 16
4.1 Entidade da *Stack* . . . . . 20
4.2 Tipo e sinal que representam a pilha . . . . . 20
4.3 Operações de *push* e *pop* e saída da *Stack* . . . . . 21
5.1 Entidade do *Prog_cnt* . . . . . 24
5.2 Sinais internos do *Prog_cnt* . . . . . 25
5.3 Parte combinacional do *Prog_cnt* . . . . . 25
5.4 Parte sequencial e saídas do *Prog_cnt* . . . . . 25
A.1 Arquivo `Trabalho2_RegBank.vhd` . . . . . 31
B.1 Arquivo `Trabalho2_RegStatus.vhd` . . . . . 32
C.1 Arquivo `Trabalho2_Stack.vhd` . . . . . 33
D.1 Arquivo `Trabalho2_ProgCnt.vhd` . . . . . 34

# SUMÁRIO

**1 Introdução . . . . . 7**
1.1 Ambiente e metodologia de simulação . . . . . 7
**2 Banco de registradores . . . . . 9**
2.1 Especificação . . . . . 9
2.2 Desenvolvimento . . . . . 9
2.3 Compilação . . . . . 12
2.4 Casos de teste e resultados . . . . . 12
**3 Registrador de status . . . . . 14**
3.1 Especificação . . . . . 14
3.2 Desenvolvimento . . . . . 14
3.3 Compilação . . . . . 16
3.4 Casos de teste e resultados . . . . . 16
**4 Pilha . . . . . 18**
4.1 Especificação . . . . . 18
4.2 Desenvolvimento . . . . . 18
4.3 Compilação . . . . . 19
4.4 Casos de teste e resultados . . . . . 19
**5 Contador de programa . . . . . 23**
5.1 Especificação . . . . . 23
5.2 Desenvolvimento . . . . . 23
5.3 Compilação . . . . . 26
5.4 Casos de teste e resultados . . . . . 26
**6 Conclusão . . . . . 28**

**Referências . . . . . 29**

**Apêndices 30**
**APÊNDICE A Código VHDL completo do Reg_bank . . . . . 31**
**APÊNDICE B Código VHDL completo do Status_reg . . . . . 32**
**APÊNDICE C Código VHDL completo da Stack . . . . . 33**
**APÊNDICE D Código VHDL completo do Prog_cnt . . . . . 34**

# 1 INTRODUÇÃO

Este trabalho tem como objetivo o desenvolvimento de quatro projetos independentes: o banco de registradores (*Reg_bank*), o registrador de status (*Status_reg*), a pilha (*Stack*) e o contador de programa (*Prog_cnt*). Cada projeto consiste em descrever o circuito em linguagem VHDL, comentar o código, simular o funcionamento e descrever os casos de teste e os resultados da simulação, de acordo com as especificações apresentadas no enunciado (GARCIA, 2026).

Diferentemente do primeiro trabalho, os quatro blocos são sequenciais: todos armazenam informação em registradores, que são escritos na borda de subida do *clock* (`clk_in`) e zerados de forma assíncrona pela entrada `nrst`, ativa em nível baixo e com preferência sobre todas as outras. O enunciado permite o uso de código concorrente ou sequencial; os registradores foram descritos com `PROCESS` e as saídas combinacionais, com atribuições concorrentes ou com um `PROCESS` combinacional. Todas as entradas e saídas são dos tipos `std_logic` ou `std_logic_vector`, e a ferramenta utilizada foi o Quartus II, versão 9.1sp2, conforme determinado.

Os conceitos de linguagem utilizados como base foram os apresentados em aula (GARCIA; MAGALHÃES, 2024; GARCIA; MAGALHÃES, 2023).

## 1.1 AMBIENTE E METODOLOGIA DE SIMULAÇÃO

Cada bloco foi desenvolvido em um projeto próprio do Quartus II 9.1sp2, para o dispositivo EP2C35F672C6 da família Cyclone II. Em todos os projetos, a entidade de nível mais alto (*top-level*) tem o mesmo nome do projeto e do arquivo VHDL: `Trabalho2_RegBank`, `Trabalho2_RegStatus`, `Trabalho2_Stack` e `Trabalho2_ProgCnt`.

A validação foi feita por simulação funcional no simulador do Quartus II, com um arquivo de formas de onda vetoriais (`.vwf`) para cada projeto. Para evitar dúvida sobre qual valor é capturado em cada borda, as entradas mudam sempre longe das bordas de subida do *clock*:

- no *Reg_bank* e no *Status_reg*, o *clock* tem período de 5 ns, com bordas de subida em 2,5 ns, 7,5 ns, 12,5 ns e assim por diante, e a simulação dura 64 ns;
- na *Stack* e no *Prog_cnt*, o *clock* tem período de 100 ns, com bordas de subida em 50 ns, 150 ns, 250 ns e assim por diante; as entradas mudam em múltiplos de 100 ns, e as simulações duram 2,8 $\mu$s e 1,2 $\mu$s, respectivamente.

Os valores esperados foram calculados a partir da especificação de cada bloco e comparados com os valores obtidos nas formas de onda. Nas tabelas de casos de teste, os barramentos de dados estão em hexadecimal e os sinais de seleção e de controle, em binário. A coluna “Instante”

indica o momento em que a saída assume o valor indicado. Para facilitar a leitura, cada figura de simulação foi dividida em duas partes, uma abaixo da outra: a de cima mostra a primeira metade do tempo de simulação e a de baixo, a segunda.

O restante do documento está organizado da seguinte forma: os Capítulos 2, 3, 4 e 5 apresentam, respectivamente, o banco de registradores, o registrador de status, a pilha e o contador de programa, cada um com sua especificação, desenvolvimento, compilação e casos de teste; e o Capítulo 6 traz as conclusões. Os códigos completos estão nos Apêndices A a D.

# 2 BANCO DE REGISTRADORES

## 2.1 ESPECIFICAÇÃO

O *Reg_bank* é um banco com quatro registradores de 8 bits, chamados R0 a R3 (GARCIA, 2026). A leitura é feita por duas saídas simultâneas, `a_do` e `b_do`, cada uma com sua entrada de seleção (`a_sel` e `b_sel`), de forma assíncrona e sem habilitação. A escrita é síncrona com a borda de subida do *clock* e só acontece quando a entrada `wr_ena` está em '1'; o registrador escrito é o selecionado por `a_sel`, e o dado vem da entrada `di`. A Tabela 1 resume as portas, e a Figura 1 mostra o diagrama funcional.

Tabela 1 – Entradas e saídas do *Reg_bank*

| Porta | Direção | Função |
|---|---|---|
| `nrst` | entrada | *Reset* assíncrono, ativo em '0'; zera R0 a R3 e tem preferência sobre as demais entradas. |
| `clk_in` | entrada | *Clock*; a escrita ocorre na borda de subida, se habilitada. |
| `di[7..0]` | entrada | Dado a ser escrito no registrador selecionado por `a_sel`. |
| `a_sel[1..0]` | entrada | Seleciona o registrador lido em `a_do` e o registrador escrito. |
| `b_sel[1..0]` | entrada | Seleciona o registrador lido em `b_do`. |
| `wr_ena` | entrada | Habilitação de escrita, ativa em '1'. |
| `a_do[7..0]` | saída | Conteúdo do registrador selecionado por `a_sel` (combinacional). |
| `b_do[7..0]` | saída | Conteúdo do registrador selecionado por `b_sel` (combinacional). |

Fonte: Adaptada de Garcia (2026).

## 2.2 DESENVOLVIMENTO

A entidade `Trabalho2_RegBank` (Código 2.1) declara as portas da Tabela 1. As entradas `di`, `a_sel`, `b_sel` e `wr_ena` recebem valores iniciais apenas para a simulação; eles não alteram o circuito sintetizado.

Os quatro registradores são os sinais `R0` a `R3`, de 8 bits. A escrita é descrita no `PROCESS registrador` (Código 2.2), sensível apenas a `nrst` e `clk_in`. O primeiro teste do `IF` é o *reset*: enquanto `nrst` vale '0', os quatro registradores são zerados, independentemente do *clock*, o que dá ao *reset* a preferência exigida. Caso contrário, na borda de subida (`RISING_EDGE`), se `wr_ena` vale '1', o `CASE` sobre `a_sel` escolhe qual registrador recebe `di`. Esse `CASE` faz o papel do decodificador da Figura 1: só o registrador selecionado é habilitado, e os outros mantêm o valor.

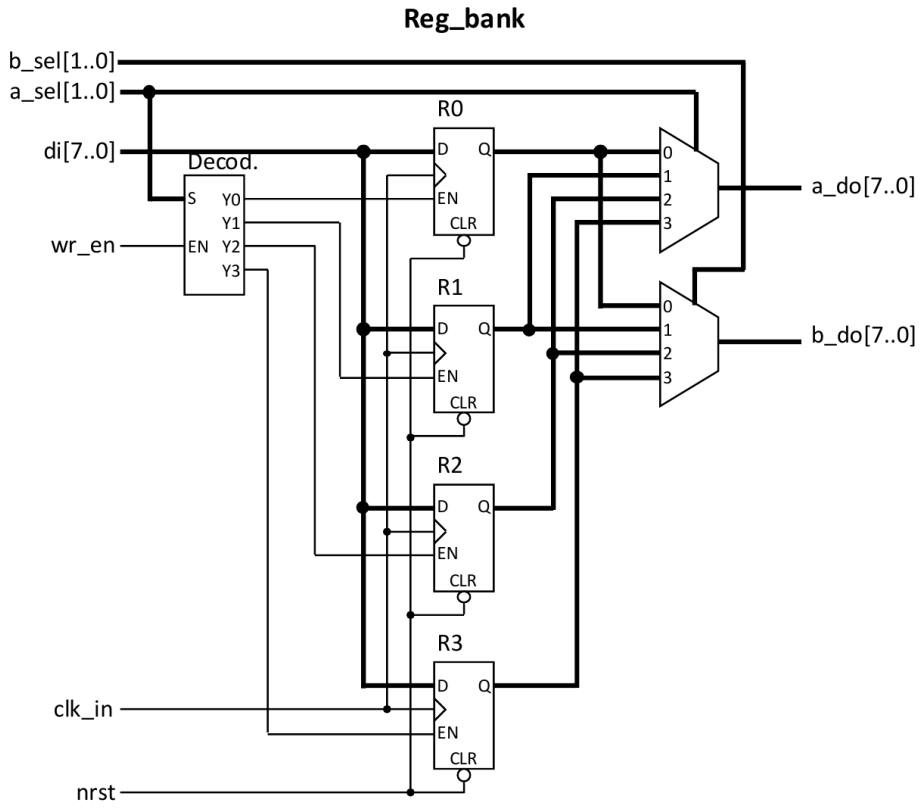
Código 2.1 – Entidade do *Reg_bank*

```
5  ENTITY Trabalho2_RegBank IS
6      PORT(
7          a_do   : OUT STD_LOGIC_VECTOR(7 downto 0); -- Saída de dado de um dos
               registradores selecionado pela entrada a_sel
8          b_do   : OUT STD_LOGIC_VECTOR(7 downto 0); -- Saída de dado de um dos
               registradores selecionado pela entrada b_sel
9          nrst   : IN STD_LOGIC; -- Entrada de reset assíncrono
10         clk_in : IN STD_LOGIC; -- Entrada de clock para escrita em todos os
               registradores
11         di     : IN STD_LOGIC_VECTOR(7 downto 0) := (others => '0'); -- Entrada
                de dados para escrita nos registradores R0 a R3
12         a_sel  : IN STD_LOGIC_VECTOR(1 downto 0) := "00"; -- Entrada para
               selecionar qual registrador vai ser lido na saída a_do
13         -- e será escrito quanto a operação de escrita estiver habilitada por
               wr_ena.
14         b_sel  : IN STD_LOGIC_VECTOR(1 downto 0) := "00"; -- Entrada para
               selecionar qual registrador vai ser lido na saída b_do
15         wr_ena : IN STD_LOGIC := '0' -- Entrada de habilitação para escrita nos
                registradores
16     );
17 END ENTITY Trabalho2_RegBank;
```

Código 2.2 – Registradores e escrita do *Reg_bank*

```
19 ARCHITECTURE Reg_Bank OF Trabalho2_RegBank IS
20
21     SIGNAL R0, R1, R2, R3 : STD_LOGIC_VECTOR (7 DOWNTO 0);
22
23 BEGIN
24
25     registrador : PROCESS(nrst, clk_in)
26     BEGIN
27         IF nrst = '0' THEN
28             R0 <= "00000000";
29             R1 <= "00000000";
30             R2 <= "00000000";
31             R3 <= "00000000";
32         ELSIF RISING_EDGE(clk_in) THEN
33             IF wr_ena = '1' THEN
34                 CASE a_sel IS
35                     WHEN "00" => R0 <= di;
36                     WHEN "01" => R1 <= di;
37                     WHEN "10" => R2 <= di;
38                     WHEN "11" => R3 <= di;
39                 END CASE;
40             END IF;
41         END IF;
42     END PROCESS registrador;
```

Figura 1 – Diagrama funcional do *Reg_bank*




Fonte: Garcia (2026).

A leitura (Código 2.3) é feita fora do `PROCESS`, com duas atribuições concorrentes `WITH...SELECT`, uma para cada saída. Elas descrevem os dois multiplexadores 4:1 do diagrama: como não dependem do *clock*, as saídas mudam assim que muda a seleção ou o conteúdo do registrador selecionado, e as duas portas podem ler registradores diferentes ao mesmo tempo.

Código 2.3 – Leitura assíncrona do *Reg_bank*

```
44      WITH a_sel SELECT
45          a_do <= R0 WHEN "00",
46                  R1 WHEN "01",
47                  R2 WHEN "10",
48                  R3 WHEN "11";
49
50      WITH b_sel SELECT
51          b_do <= R0 WHEN "00",
52                  R1 WHEN "01",
53                  R2 WHEN "10",
54                  R3 WHEN "11";
55
56  END ARCHITECTURE Reg_Bank;
```

## 2.3 COMPILAÇÃO

A compilação terminou com sucesso (Tabela 2). O circuito usa 32 registradores, exatamente os $4 \times 8$ bits de R0 a R3, e 31 pinos, que correspondem às portas da entidade (1 + 1 + 8 + 2 + 2 + 1 de entrada e 8 + 8 de saída).

Tabela 2 – Resumo da compilação do bloco *Reg_bank* no Quartus II 9.1sp2

| Item do *Flow Summary* | Valor |
|---|---|
| *Flow Status* | *Successful* |
| Dispositivo | EP2C35F672C6 (Cyclone II) |
| Elementos lógicos (*total logic elements*) | 36 / 33.216 ($< 1\%$) |
| Funções combinacionais | 36 |
| Registradores (*dedicated logic registers*) | 32 |
| Pinos | 31 / 475 |

Fonte: Elaborada pelos autores (2026), a partir do relatório de compilação do Quartus II.

## 2.4 CASOS DE TESTE E RESULTADOS

A Tabela 3 resume os casos de teste e a Figura 2 mostra a simulação. Durante os primeiros 2 ns, `nrst` vale ‘0’ e todos os registradores ficam zerados (caso 1). Na borda de 2,5 ns, R0 recebe `di` = 00h (caso 2); na de 7,5 ns, `wr_ena` vale ‘0’ e nada é escrito (caso 3). Na borda de 12,5 ns, o valor 4Ah é escrito em R0 e aparece nas duas saídas, pois `a_sel` e `b_sel` selecionam R0 (caso 4).

Em 14 ns, sem borda de *clock*, `b_sel` passa a 11 e `b_do` mostra imediatamente o conteúdo de R3 (00h), enquanto `a_do` continua mostrando R0 (4Ah). Esse caso confirma que a leitura é assíncrona e que as duas saídas leem registradores diferentes ao mesmo tempo (caso 5). Na borda de 17,5 ns, o valor DEh sobrescreve R0 (caso 6), e em 18 ns, de novo sem borda, a volta de `b_sel` para 00 faz `b_do` mostrar DEh (caso 7). A partir da borda de 22,5 ns, R0 recebe 00h a cada borda enquanto `wr_ena` vale ‘1’, e as saídas permanecem em 00h até o fim da simulação (casos 8 e 9).

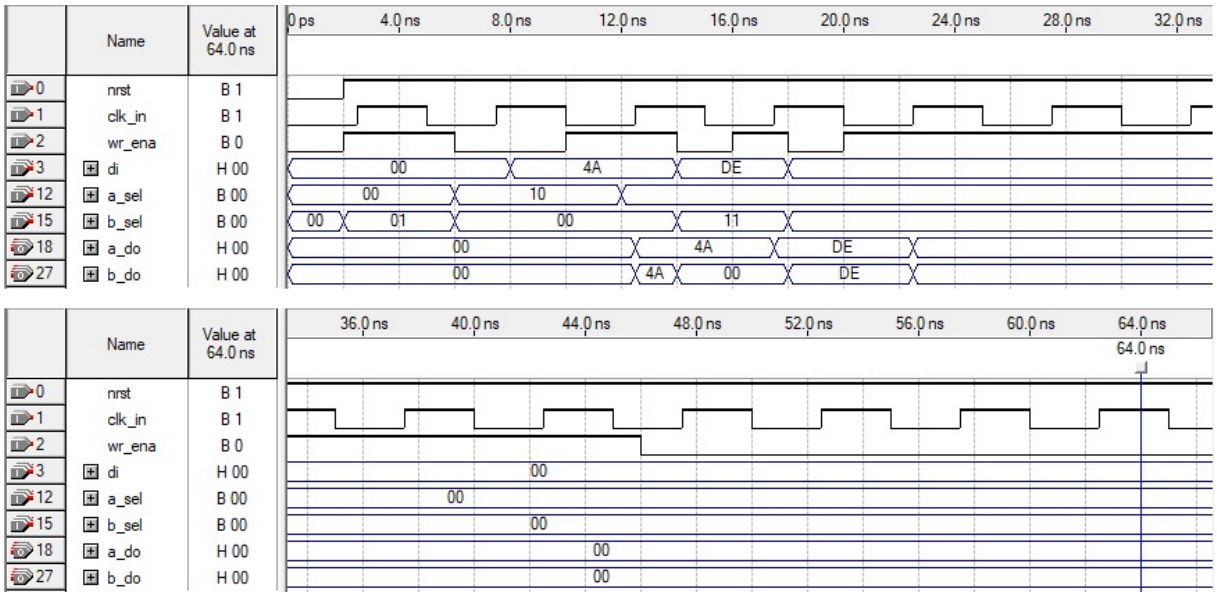
Os valores obtidos coincidem com os esperados em todos os casos.

Tabela 3 – Casos de teste do *Reg_bank*

| | | | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|---|---|
| Nº | Instante (ns) | wr_ena | a_sel | b_sel | di | a_do | b_do | a_do | b_do |
| 1 | 0 (`nrst` = 0) | 0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 2 | 2,5 (borda) | 1 | 00 | 01 | 00 | 00 | 00 | 00 | 00 |
| 3 | 7,5 (borda) | 0 | 10 | 00 | 00 | 00 | 00 | 00 | 00 |
| 4 | 12,5 (borda) | 1 | 00 | 00 | 4A | 4A | 4A | 4A | 4A |
| 5 | 14 (sem borda) | 0 | 00 | 11 | DE | 4A | 00 | 4A | 00 |
| 6 | 17,5 (borda) | 1 | 00 | 11 | DE | DE | 00 | DE | 00 |
| 7 | 18 (sem borda) | 0 | 00 | 00 | 00 | DE | DE | DE | DE |
| 8 | 22,5 (borda) | 1 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |
| 9 | 27,5 a 64 | 1/0 | 00 | 00 | 00 | 00 | 00 | 00 | 00 |

Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (di, a_do e b_do) e binário (wr_ena, a_sel e b_sel).

Figura 2 – Simulação do *Reg_bank*




Fonte: Elaborada pelos autores (2026).

# 3 REGISTRADOR DE STATUS

## 3.1 ESPECIFICAÇÃO

O *Status_reg* tem cinco registradores independentes de 1 bit, chamados RN, RV, RZ, RH e RC, que armazenam os sinalizadores gerados pela ULA do primeiro trabalho (GARCIA, 2026). As entradas, as saídas e as habilitações de escrita são vetores de 5 bits, e cada bit corresponde a um registrador (Tabela 4). Cada registrador só é escrito, na borda de subida do *clock*, quando o seu bit de habilitação vale '1'. A saída `status_reg_out` mostra sempre o conteúdo dos registradores, sem depender do *clock* nem de habilitação. As portas estão na Tabela 5 e o diagrama funcional, na Figura 3.

Tabela 4 – Correspondência entre os bits e os registradores do *Status_reg*

| Bit | Registrador | Nome | Sinal da ULA armazenado |
|---|---|---|---|
| 0 | RN | *negative* | `n_flag` |
| 1 | RV | *overflow* | `v_flag` |
| 2 | RZ | *zero* | `z_flag` |
| 3 | RH | *half carry/borrow* | `h_flag` |
| 4 | RC | *carry/borrow* | `c_flag` |

Fonte: Adaptada de Garcia (2026).

Tabela 5 – Entradas e saídas do *Status_reg*

| Porta | Direção | Função |
|---|---|---|
| `nrst` | entrada | *Reset* assíncrono, ativo em '0'; zera os cinco registradores e tem preferência sobre as demais entradas. |
| `clk_in` | entrada | *Clock*; a escrita ocorre na borda de subida, se habilitada. |
| `status_reg_in[4..0]` | entrada | Dados a serem escritos nos registradores (Tabela 4). |
| `status_reg_wr_ena[4..0]` | entrada | Habilitação de escrita de cada registrador, ativa em '1', na mesma ordem de bits. |
| `status_reg_out[4..0]` | saída | Conteúdo dos registradores, na mesma ordem de bits. |

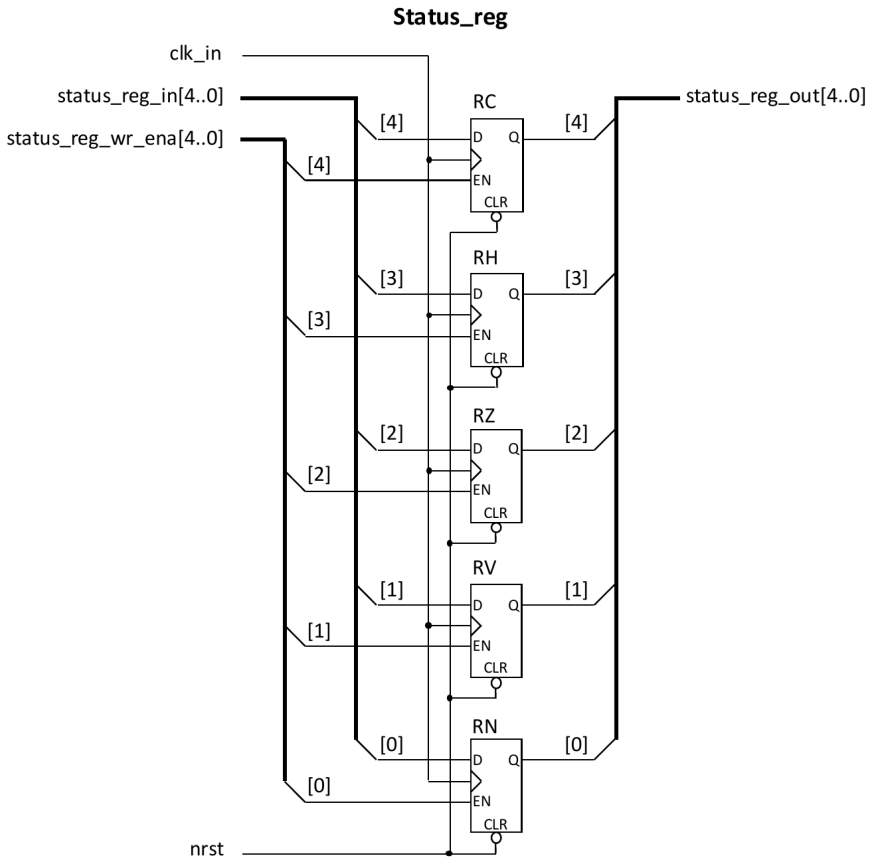
Fonte: Adaptada de Garcia (2026).

## 3.2 DESENVOLVIMENTO

O Código 3.1 mostra a entidade `Trabalho2_RegStatus`.

Os cinco registradores foram reunidos em um único sinal de 5 bits, `registradores`, com a mesma ordem de bits das portas (Código 3.2). No `PROCESS Reg_Status`, o *reset* assíncrono

Figura 3 – Diagrama funcional do *Status_reg*




Fonte: Garcia (2026).

zera o vetor inteiro. Na borda de subida do *clock*, um laço `FOR` percorre os bits de 0 a 4 e copia `status_reg_in(i)` para `registradores(i)` somente quando `status_reg_wr_ena(i)` vale '1'. Na síntese, o laço é desdobrado em cinco *flip-flops* com habilitação própria, como no diagrama da Figura 3: um bit não habilitado simplesmente mantém o valor anterior. A saída é ligada ao

Código 3.1 – Entidade do *Status_reg*

```
5  ENTITY Trabalho2_RegStatus IS
6      PORT(
7          nrst : IN STD_LOGIC; -- Entrada de reset assíncrono
8          clk_in : IN STD_LOGIC; -- Entrada de clock para escrita no registrador
9          status_reg_in : IN STD_LOGIC_VECTOR (4 DOWNTO 0); -- Entrada de dado
               para escrita nos registradores [0]=RN [1]=RV [2]=RZ [3]=RH [4]=RC
10         status_reg_wr_ena : IN STD_LOGIC_VECTOR (4 DOWNTO 0); -- Entrada de
               habilitação para escrita registradores
11         status_reg_out : OUT STD_LOGIC_VECTOR (4 DOWNTO 0) -- Saída dos
               registradores RN, RV, RZ, RH e RC
12     );
13 END ENTITY Trabalho2_RegStatus;
```

vetor por uma atribuição concorrente, fora do `PROCESS`.

Código 3.2 – Arquitetura do *Status_reg*

```
15 ARCHITECTURE Reg_Status OF Trabalho2_RegStatus IS
16     SIGNAL registradores: STD_LOGIC_VECTOR (4 DOWNTO 0); -- Vetor que faz o
           papel dos registradores, [0]=RN [1]=RV [2]=RZ [3]=RH [4]=RC
17 BEGIN
18     Reg_Status : PROCESS(nrst, clk_in)
19     BEGIN
20         IF nrst = '0' THEN
21             registradores <= (OTHERS => '0');
22         ELSIF RISING_EDGE (clk_in) THEN
23             FOR i IN 0 to 4 LOOP
24                 IF status_reg_wr_ena(i) = '1' THEN
25                     registradores(i) <= status_reg_in(i);
26                 END IF;
27             END LOOP;
28         END IF;
29     END PROCESS Reg_Status;
30
31     status_reg_out <= registradores;
32
33 END ARCHITECTURE Reg_Status;
```

## 3.3 COMPILAÇÃO

A compilação terminou com sucesso (Tabela 6). O circuito usa 5 registradores, um para cada sinalizador, e 17 pinos (1 + 1 + 5 + 5 de entrada e 5 de saída).

Tabela 6 – Resumo da compilação do bloco *Status_reg* no Quartus II 9.1sp2

| Item do *Flow Summary* | Valor |
|---|---|
| *Flow Status* | *Successful* |
| Dispositivo | EP2C35F672C6 (Cyclone II) |
| Elementos lógicos (*total logic elements*) | 5 / 33.216 ($<$ 1%) |
| Funções combinacionais | 5 |
| Registradores (*dedicated logic registers*) | 5 |
| Pinos | 17 / 475 |

Fonte: Elaborada pelos autores (2026), a partir do relatório de compilação do Quartus II.

## 3.4 CASOS DE TESTE E RESULTADOS

A Tabela 7 resume os casos de teste e a Figura 4 mostra a simulação. Os vetores estão em binário, na ordem RC, RH, RZ, RV, RN (bits 4 a 0). No caso 1, `nrst` vale ‘0’ durante os primeiros 2 ns e zera os registradores; na borda de 2,5 ns, nenhum bit está habilitado e a saída

continua 00000. Os demais casos foram escolhidos para mostrar que cada registrador obedece apenas ao seu próprio bit de habilitação:

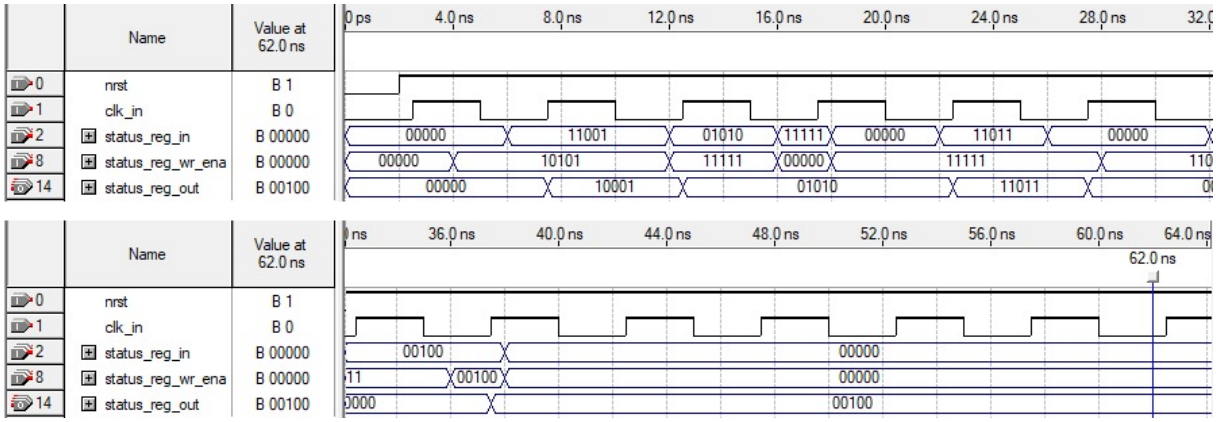
- no caso 2, com habilitação 10101, só RC, RZ e RN são escritos: a entrada 11001 resulta em 10001, pois RH e RV mantêm o valor 0;
- no caso 4, com habilitação 00000, a entrada 11111 é ignorada e a saída mantém 01010;
- no caso 7, a entrada tem o bit de RZ em '1', mas esse bit não está habilitado (11011), e a saída continua 00000;
- no caso 8, apenas RZ é habilitado (00100) e passa a valer '1'.

Tabela 7 – Casos de teste do *Status_reg*

| | | | | Esperado | Obtido |
|---|---|---|---|---|---|
| Nº | Instante (ns) | status_reg_in | status_reg_wr_ena | status_reg_out | status_reg_out |
| 1 | 0 e 2,5 | 00000 | 00000 | 00000 | 00000 |
| 2 | 7,5 (borda) | 11001 | 10101 | 10001 | 10001 |
| 3 | 12,5 (borda) | 01010 | 11111 | 01010 | 01010 |
| 4 | 17,5 (borda) | 11111 | 00000 | 01010 | 01010 |
| 5 | 22,5 (borda) | 11011 | 11111 | 11011 | 11011 |
| 6 | 27,5 (borda) | 00000 | 11111 | 00000 | 00000 |
| 7 | 32,5 (borda) | 00100 | 11011 | 00000 | 00000 |
| 8 | 37,5 (borda) | 00100 | 00100 | 00100 | 00100 |
| 9 | 42,5 a 64 | 00000 | 00000 | 00100 | 00100 |

Fonte: Elaborada pelos autores (2026). Valores em binário, na ordem dos bits [4..0] = RC, RH, RZ, RV, RN.

Figura 4 – Simulação do *Status_reg*




Fonte: Elaborada pelos autores (2026).

Os valores obtidos coincidem com os esperados em todos os casos.

# 4 PILHA

## 4.1 ESPECIFICAÇÃO

A *Stack* é uma pilha com oito registradores de 11 bits (GARCIA, 2026). Na operação *push*, o valor de `stack_in` é escrito no primeiro registrador (topo) e cada registrador recebe o conteúdo do anterior, até o oitavo, que recebe o conteúdo do sétimo. Na operação *pop*, cada registrador recebe o conteúdo do seguinte, e o oitavo recebe zero. As duas operações acontecem na borda de subida do *clock*. Se `stack_push` e `stack_pop` forem ativados ao mesmo tempo, o *push* tem preferência. Mais de oito operações *push* seguidas fazem o conteúdo mais antigo ser perdido (*stack overflow*). A saída `stack_out` mostra sempre o topo da pilha. As portas estão na Tabela 8 e o diagrama funcional, na Figura 5.

Tabela 8 – Entradas e saídas da *Stack*

| Porta | Direção | Função |
|---|---|---|
| `nrst` | entrada | *Reset* assíncrono, ativo em '0'; zera os oito registradores e tem preferência sobre as demais entradas. |
| `clk_in` | entrada | *Clock*; *push* e *pop* ocorrem na borda de subida. |
| `stack_in[10..0]` | entrada | Valor a ser colocado na pilha. |
| `stack_push` | entrada | Habilita o *push*, ativo em '1'; tem preferência sobre o *pop*. |
| `stack_pop` | entrada | Habilita o *pop*, ativo em '1'. |
| `stack_out[10..0]` | saída | Conteúdo do topo da pilha (primeiro registrador). |

Fonte: Adaptada de Garcia (2026).

## 4.2 DESENVOLVIMENTO

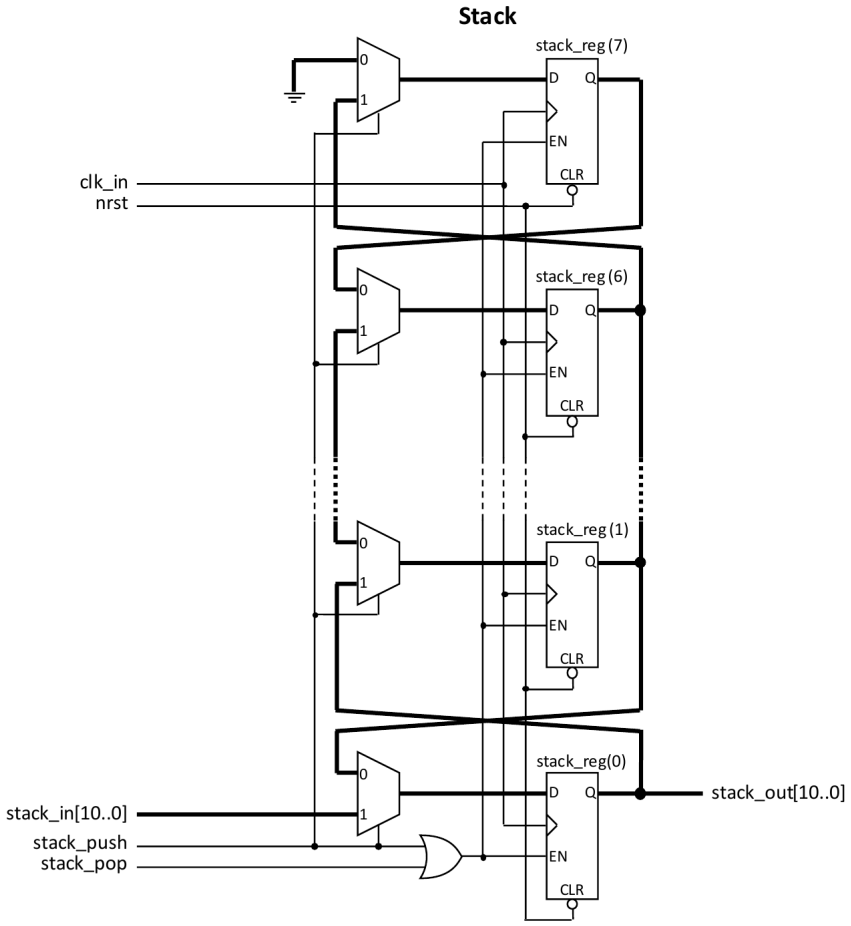
O Código 4.1 mostra a entidade `Trabalho2_Stack`.

Para representar os oito registradores, foi criado o tipo `tipo_pilha`, um vetor de oito posições de 11 bits, e o sinal `pilha`, desse tipo (Código 4.2). A posição `pilha(0)` é o topo, e `pilha(7)`, o oitavo registrador.

O `PROCESS Stack` (Código 4.3) segue a mesma estrutura dos registradores anteriores: primeiro o *reset* assíncrono, que zera todas as posições, e depois a borda de subida do *clock*. A preferência do *push* é obtida pela ordem do `IF...ELSIF`: o *pop* só é avaliado quando `stack_push` vale '0'.

No *push*, `pilha(0)` recebe `stack_in` e um laço `FOR` faz cada posição `i` receber a posição `i-1`. No *pop*, o laço faz cada posição `i` receber a posição `i+1`, e `pilha(7)` recebe zero. Esses laços funcionam como um registrador de deslocamento porque, dentro de um `PROCESS`, os sinais

Figura 5 – Diagrama funcional da *Stack*




Fonte: Garcia (2026).

só são atualizados ao final da execução: todas as atribuições usam os valores que a pilha tinha antes da borda, e o deslocamento ocorre de uma só vez, como no diagrama da Figura 5. Quando nenhuma das duas entradas está ativa, nenhum sinal é atribuído e a pilha mantém o conteúdo.

## 4.3 COMPILAÇÃO

A compilação terminou com sucesso (Tabela 9). O circuito usa 88 registradores, exatamente os $8 \times 11$ bits da pilha, e 26 pinos (1 + 1 + 11 + 1 + 1 de entrada e 11 de saída).

## 4.4 CASOS DE TESTE E RESULTADOS

A Tabela 10 resume os casos de teste e a Figura 6 mostra a simulação. Os casos 2 e 3 mostram três operações *push* (001h, 002h e 003h) seguidas de três operações *pop*, que devolvem os valores na ordem inversa, até 000h. Nos casos 4 a 6 são feitas nove operações *push* seguidas

Código 4.1 – Entidade da *Stack*

```
5  ENTITY Trabalho2_Stack IS
6      PORT(
7          nrst       : IN STD_LOGIC; -- Entrada de reset assincrono
8          clk_in     : IN STD_LOGIC; -- Entrada de clock para escrita nos
               registradores
9          stack_in   : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); --
               Entrada para a pilha
10         stack_push : IN STD_LOGIC := '0'; -- Entrada de habilitação para
               colocar valores na pilha (tem preferência sobre o pop)
11         stack_pop  : IN STD_LOGIC := '0'; -- Entrada de habilitação para
               retirar valores da pilha
12         stack_out  : OUT STD_LOGIC_VECTOR(10 DOWNTO 0) -- Saída correspondente
               à primeira posição (topo) da pilha
13     );
14 END ENTITY Trabalho2_Stack;
```

Código 4.2 – Tipo e sinal que representam a pilha

```
16 ARCHITECTURE Stack OF Trabalho2_Stack IS
17
18     -- Tipo que representa os 8 registradores de 11 bits da pilha, pilha(0) é o
           topo
19     TYPE tipo_pilha IS ARRAY (0 TO 7) OF STD_LOGIC_VECTOR(10 DOWNTO 0);
20     SIGNAL pilha : tipo_pilha;
```

Tabela 9 – Resumo da compilação do bloco *Stack* no Quartus II 9.1sp2

| Item do *Flow Summary* | Valor |
|---|---|
| *Flow Status* | *Successful* |
| Dispositivo | EP2C35F672C6 (Cyclone II) |
| Elementos lógicos (*total logic elements*) | 89 / 33.216 ($< 1\%$) |
| Funções combinacionais | 89 |
| Registradores (*dedicated logic registers*) | 88 |
| Pinos | 26 / 475 |

Fonte: Elaborada pelos autores (2026), a partir do relatório de compilação do Quartus II.

(001h a 009h), uma a mais que a capacidade da pilha: nas oito operações *pop* seguintes, a saída vai de 008h a 002h e termina em 000h, pois o valor 001h foi descartado do oitavo registrador (*stack overflow*).

No caso 7, `stack_push` e `stack_pop` são ativados ao mesmo tempo, com `stack_in` = 7FFh, e o valor vai para o topo, o que confirma a preferência do *push*. No caso 8, com as duas entradas em ‘0’, a pilha mantém o conteúdo. Por fim, no caso 9, `nrst` vai a ‘0’ em 2.600 ns e a saída é zerada imediatamente, antes da borda de 2.650 ns, o que confirma que o *reset* é assíncrono.

Código 4.3 – Operações de *push* e *pop* e saída da *Stack*

```
24      Stack : PROCESS(nrst, clk_in)
25      BEGIN
26          IF nrst = '0' THEN
27              pilha <= (OTHERS => (OTHERS => '0'));
28          ELSIF RISING_EDGE(clk_in) THEN
29              IF stack_push = '1' THEN
30                  -- push: o valor de entrada vai para o topo e cada registrador
                       recebe o conteudo do anterior
31                  -- o conteudo do oitavo registrador e perdido (stack overflow)
32                  pilha(0) <= stack_in;
33                  FOR i IN 1 TO 7 LOOP
34                      pilha(i) <= pilha(i-1);
35                  END LOOP;
36              ELSIF stack_pop = '1' THEN
37                  -- pop: cada registrador recebe o conteudo do seguinte e o
                       oitavo recebe zero
38                  FOR i IN 0 TO 6 LOOP
39                      pilha(i) <= pilha(i+1);
40                  END LOOP;
41                  pilha(7) <= (OTHERS => '0');
42              END IF;
43          END IF;
44      END PROCESS Stack;
45
46      stack_out <= pilha(0);
47
48  END ARCHITECTURE Stack;
```

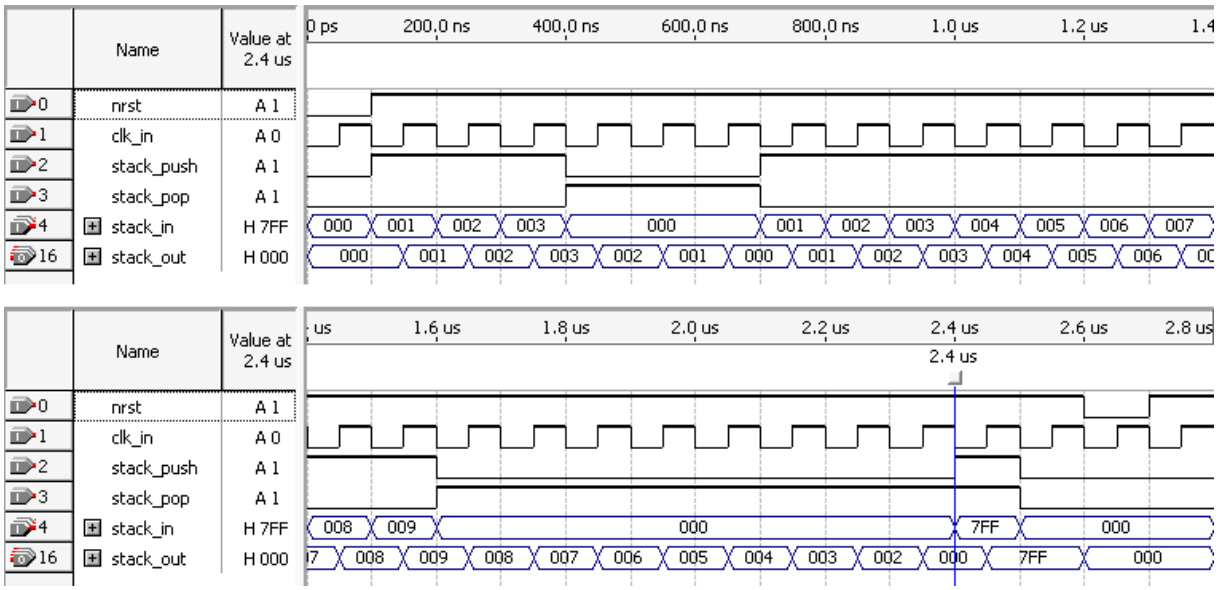
Tabela 10 – Casos de teste da *Stack*

| | | | | Esperado | Obtido |
|---|---|---|---|---|---|
| Nº | Instante (ns) | Operação | stack_in | stack_out | stack_out |
| 1 | 0 | *reset* (nrst = 0) | 000 | 000 | 000 |
| 2 | 150, 250, 350 | 3 × *push* | 001, 002, 003 | 001, 002, 003 | 001, 002, 003 |
| 3 | 450, 550, 650 | 3 × *pop* | – | 002, 001, 000 | 002, 001, 000 |
| 4 | 750 a 1.550 | 9 × *push* | 001 a 009 | 001 a 009 | 001 a 009 |
| 5 | 1.650 a 2.250 | 7 × *pop* | – | 008 a 002 | 008 a 002 |
| 6 | 2.350 | 8.º *pop* | – | 000 | 000 |
| 7 | 2.450 | *push* e *pop* juntos | 7FF | 7FF | 7FF |
| 8 | 2.550 | nenhuma operação | – | 7FF | 7FF |
| 9 | 2.600 (sem borda) | *reset* (nrst = 0) | – | 000 | 000 |

Fonte: Elaborada pelos autores (2026). Valores em hexadecimal.

Os valores obtidos coincidem com os esperados em todos os casos.

Figura 6 – Simulação da *Stack*




Fonte: Elaborada pelos autores (2026).

# 5 CONTADOR DE PROGRAMA

## 5.1 ESPECIFICAÇÃO

O *Prog_cnt* é um contador síncrono módulo 2048 (de 0 a 2047), com *reset* assíncrono e carga síncrona (GARCIA, 2026). Ele tem duas saídas: `pc_out`, que é a saída dos *flip-flops* do contador, e `next_pc_out`, que é a entrada desses *flip-flops*, ou seja, o valor que será carregado na próxima borda de subida do *clock*. A entrada `pc_ctrl` seleciona a função do contador, conforme a Tabela 11. As portas estão na Tabela 12 e o diagrama funcional, na Figura 7. O enunciado recomenda que o contador seja descrito em dois `PROCESS`, um com a parte combinacional e outro com a parte sequencial.

Tabela 11 – Funções selecionadas por `pc_ctrl`

| `pc_ctrl` | Função |
|---|---|
| 00 | Permanece como está |
| 01 | Carrega o valor do topo da pilha (`from_stack`) |
| 10 | Carrega um novo valor (`new_pc_in`) |
| 11 | Incrementa o contador (+1) |

Fonte: Adaptada de Garcia (2026).

Tabela 12 – Entradas e saídas do *Prog_cnt*

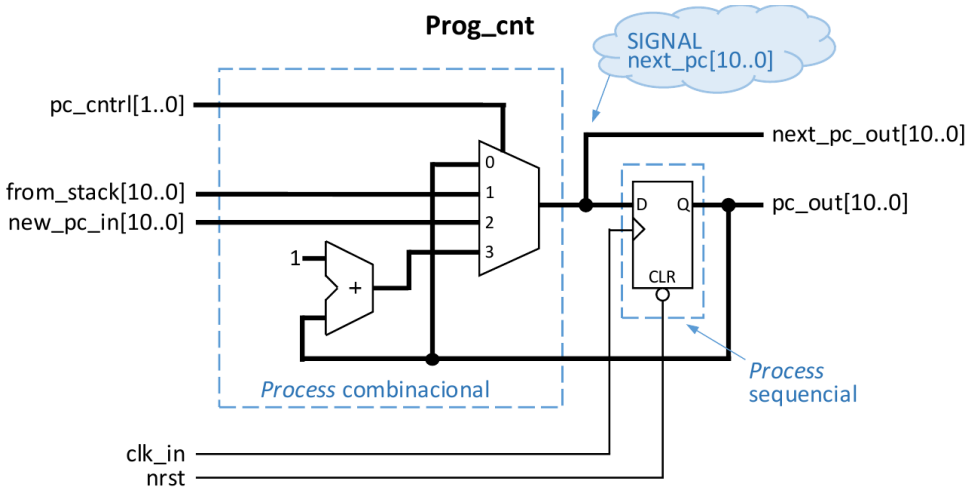
| Porta | Direção | Função |
|---|---|---|
| `nrst` | entrada | *Reset* assíncrono, ativo em ‘0’; zera o contador e tem preferência sobre as demais entradas. |
| `clk_in` | entrada | *Clock*; o incremento e as cargas ocorrem na borda de subida. |
| `pc_ctrl[1..0]` | entrada | Controle do contador (Tabela 11). |
| `new_pc_in[10..0]` | entrada | Valor carregado quando `pc_ctrl` = 10. |
| `from_stack[10..0]` | entrada | Valor carregado quando `pc_ctrl` = 01. |
| `next_pc_out[10..0]` | saída | Próximo valor do contador (combinacional). |
| `pc_out[10..0]` | saída | Valor atual do contador. |

Fonte: Adaptada de Garcia (2026).

## 5.2 DESENVOLVIMENTO

O Código 5.1 mostra a entidade `Trabalho2_ProgCnt`.

Figura 7 – Diagrama funcional do *Prog_cnt*




Fonte: Garcia (2026).

Código 5.1 – Entidade do *Prog_cnt*

```
5  ENTITY Trabalho2_ProgCnt IS
6      PORT(
7          nrst        : IN STD_LOGIC; -- Entrada de reset assíncrono
8          clk_in      : IN STD_LOGIC; -- Entrada de clock para incremento ou
               carga do contador
9          pc_ctrl     : IN STD_LOGIC_VECTOR(1 DOWNTO 0) := "00"; -- Controle: 00
               mantém, 01 carrega from_stack, 10 carrega new_pc_in, 11 incrementa
10         new_pc_in   : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); --
               Entrada para carga no contador quando pc_ctrl = "10"
11         from_stack  : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); --
               Entrada para carga no contador quando pc_ctrl = "01"
12         next_pc_out : OUT STD_LOGIC_VECTOR(10 DOWNTO 0); -- Saída do próximo
               valor do contador (combinacional)
13         pc_out      : OUT STD_LOGIC_VECTOR(10 DOWNTO 0) -- Saída do contador
14     );
15 END ENTITY Trabalho2_ProgCnt;
```

Seguindo a recomendação do enunciado e o diagrama da Figura 7, o contador foi dividido em duas partes, ligadas por dois sinais internos de 11 bits: `pc`, a saída dos *flip-flops*, e `next_pc`, a entrada deles (Código 5.2).

O `PROCESS Combinacional` (Código 5.3) descreve o multiplexador do diagrama. A lista de sensibilidade contém todas as entradas usadas no cálculo, inclusive `pc`, para que `next_pc` seja recalculado sempre que qualquer uma delas mudar. O `CASE` sobre `pc_ctrl` escolhe entre `from_stack`, `new_pc_in`, `pc + 1` e o próprio `pc`. A cláusula `WHEN OTHERS` cobre o valor 00 (manter) e os demais valores possíveis do tipo `std_logic`, e garante que `next_pc` receba um valor em todos os caminhos, o que evita a criação de *latches*. O incremento é feito com conversão para `UNSIGNED`; como o vetor tem 11 bits, o valor 2047 + 1 volta naturalmente a 0, o que implementa

Código 5.2 – Sinais internos do *Prog_cnt*

```
17 ARCHITECTURE Prog_Cnt OF Trabalho2_ProgCnt IS
18
19     SIGNAL pc      : STD_LOGIC_VECTOR(10 DOWNTO 0); -- Saída dos flip-flops do
           contador
20     SIGNAL next_pc : STD_LOGIC_VECTOR(10 DOWNTO 0); -- Entrada dos flip-flops
           do contador
```

a contagem módulo 2048 sem nenhum teste adicional.

Código 5.3 – Parte combinacional do *Prog_cnt*

```
24     -- Parte combinacional: calcula o valor a ser carregado na próxima borda de
           subida do clock
25     -- como o vetor tem 11 bits, o incremento de 2047 volta naturalmente para 0
           (módulo 2048)
26     Combinacional : PROCESS(pc_ctrl, pc, new_pc_in, from_stack)
27     BEGIN
28         CASE pc_ctrl IS
29             WHEN "01"   => next_pc <= from_stack;
30             WHEN "10"   => next_pc <= new_pc_in;
31             WHEN "11"   => next_pc <= STD_LOGIC_VECTOR(UNSIGNED(pc) + 1);
32             WHEN OTHERS => next_pc <= pc;
33         END CASE;
34     END PROCESS Combinacional;
```

O PROCESS Sequencial (Código 5.4) contém apenas os *flip-flops*: o *reset* assíncrono zera pc, e cada borda de subida copia next_pc para pc. Como a escolha da função já foi feita na parte combinacional, todas as cargas e o incremento são síncronos. As saídas são ligadas aos sinais internos por atribuições concorrentes.

Código 5.4 – Parte sequencial e saídas do *Prog_cnt*

```
36     -- Parte sequencial: registra o próximo valor na borda de subida do clock
37     Sequencial : PROCESS(nrst, clk_in)
38     BEGIN
39         IF nrst = '0' THEN
40             pc <= (OTHERS => '0');
41         ELSIF RISING_EDGE(clk_in) THEN
42             pc <= next_pc;
43         END IF;
44     END PROCESS Sequencial;
45
46     next_pc_out <= next_pc;
47     pc_out <= pc;
48
49 END ARCHITECTURE Prog_Cnt;
```

## 5.3 COMPILAÇÃO

A compilação terminou com sucesso (Tabela 13). O circuito usa 11 registradores, um para cada bit do contador, o que confirma que `next_pc` é puramente combinacional e não gerou registradores. Os 48 pinos correspondem às portas da entidade (1 + 1 + 2 + 11 + 11 de entrada e 11 + 11 de saída).

Tabela 13 – Resumo da compilação do bloco *Prog_cnt* no Quartus II 9.1sp2

| Item do *Flow Summary* | Valor |
|---|---|
| *Flow Status* | *Successful* |
| Dispositivo | EP2C35F672C6 (Cyclone II) |
| Elementos lógicos (*total logic elements*) | 34 / 33.216 ($< 1\%$) |
| Funções combinacionais | 33 |
| Registradores (*dedicated logic registers*) | 11 |
| Pinos | 48 / 475 |

Fonte: Elaborada pelos autores (2026), a partir do relatório de compilação do Quartus II.

## 5.4 CASOS DE TESTE E RESULTADOS

Na simulação, `new_pc_in` vale 7FEh e `from_stack` vale 123h durante todo o tempo. Assim, ela também mostra que essas entradas só são carregadas quando `pc_ctrl` as seleciona. A Tabela 14 resume os casos de teste e a Figura 8 mostra a simulação.

Os casos 2 a 5 mostram o incremento. Em 100 ns, quando `pc_ctrl` passa a 11, `next_pc_out` muda imediatamente para 001h, antes de qualquer borda, porque é combinacional; `pc_out` só assume esse valor na borda seguinte. Nos casos 6 e 7, com `pc_ctrl` = 00, o contador mantém o valor mesmo com a borda de 450 ns. Nos casos 8 e 9, o valor 7FEh de `new_pc_in` aparece primeiro em `next_pc_out` e é carregado na borda de 550 ns.

Os casos 10 a 12 mostram a contagem módulo 2048: o contador vai de 7FEh a 7FFh e, na borda seguinte, volta a 000h. Nos casos 13 e 14, o valor 123h de `from_stack` é carregado. Por fim, no caso 17, `nrst` vai a '0' em 1.000 ns e `pc_out` é zerado imediatamente, sem esperar a borda; a borda de 1.050 ns, ainda durante o *reset*, é ignorada (caso 18).

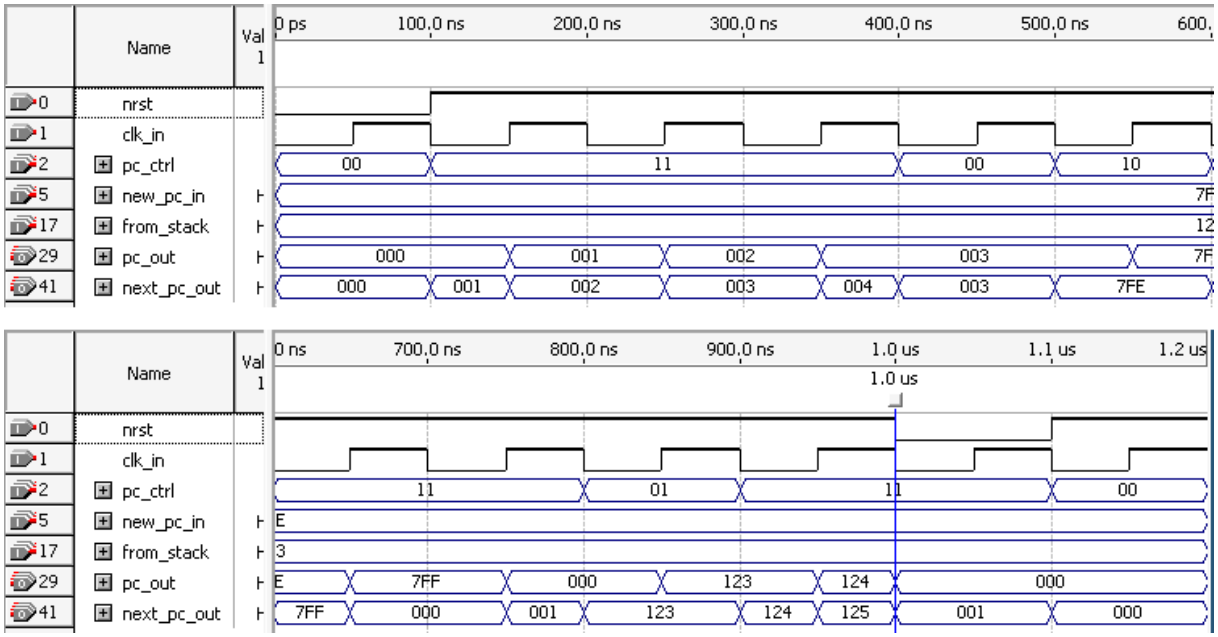
Os valores obtidos coincidem com os esperados em todos os casos.

Tabela 14 – Casos de teste do *Prog_cnt*

| | | | | Esperado | | Obtido | |
|---|---|---|---|---|---|---|---|
| Nº | Instante (ns) | nrst | pc_ctrl | next_pc_out | pc_out | next_pc_out | pc_out |
| 1 | 0 | 0 | 00 | 000 | 000 | 000 | 000 |
| 2 | 100 (sem borda) | 1 | 11 | 001 | 000 | 001 | 000 |
| 3 | 150 (borda) | 1 | 11 | 002 | 001 | 002 | 001 |
| 4 | 250 (borda) | 1 | 11 | 003 | 002 | 003 | 002 |
| 5 | 350 (borda) | 1 | 11 | 004 | 003 | 004 | 003 |
| 6 | 400 (sem borda) | 1 | 00 | 003 | 003 | 003 | 003 |
| 7 | 450 (borda) | 1 | 00 | 003 | 003 | 003 | 003 |
| 8 | 500 (sem borda) | 1 | 10 | 7FE | 003 | 7FE | 003 |
| 9 | 550 (borda) | 1 | 10 | 7FE | 7FE | 7FE | 7FE |
| 10 | 600 (sem borda) | 1 | 11 | 7FF | 7FE | 7FF | 7FE |
| 11 | 650 (borda) | 1 | 11 | 000 | 7FF | 000 | 7FF |
| 12 | 750 (borda) | 1 | 11 | 001 | 000 | 001 | 000 |
| 13 | 800 (sem borda) | 1 | 01 | 123 | 000 | 123 | 000 |
| 14 | 850 (borda) | 1 | 01 | 123 | 123 | 123 | 123 |
| 15 | 900 (sem borda) | 1 | 11 | 124 | 123 | 124 | 123 |
| 16 | 950 (borda) | 1 | 11 | 125 | 124 | 125 | 124 |
| 17 | 1.000 (sem borda) | 0 | 11 | 001 | 000 | 001 | 000 |
| 18 | 1.050 (borda) | 0 | 11 | 001 | 000 | 001 | 000 |
| 19 | 1.100 (sem borda) | 1 | 00 | 000 | 000 | 000 | 000 |
| 20 | 1.150 (borda) | 1 | 00 | 000 | 000 | 000 | 000 |

Fonte: Elaborada pelos autores (2026). Valores em hexadecimal (next_pc_out e pc_out) e binário (nrst e pc_ctrl). As entradas new_pc_in = 7FEh e from_stack = 123h são constantes.

Figura 8 – Simulação do *Prog_cnt*




Fonte: Elaborada pelos autores (2026).

# 6 CONCLUSÃO

Foram desenvolvidos em VHDL os quatro blocos sequenciais especificados: o banco de registradores, o registrador de status, a pilha e o contador de programa, cada um em um projeto próprio do Quartus II 9.1sp2 e com portas somente dos tipos `std_logic` e `std_logic_vector`, como exigido.

Os quatro blocos seguem a mesma estrutura: um `PROCESS` sensível apenas a `nrst` e `clk_in`, com o *reset* assíncrono testado antes da borda de subida, o que garante a preferência do *reset* sobre as demais entradas. As partes combinacionais (os multiplexadores de leitura do *Reg_bank*, as saídas e o cálculo do próximo valor do *Prog_cnt*) ficaram fora desse `PROCESS`, em atribuições concorrentes ou em um `PROCESS` combinacional. Os relatórios de compilação confirmaram essa estrutura: o número de registradores de cada bloco é exatamente o previsto pela especificação (32, 5, 88 e 11).

As simulações realizadas apresentaram resultados iguais aos esperados em todos os casos de teste.

# REFERÊNCIAS

GARCIA, F. *Especificações para o segundo trabalho*. 2026. Material didático. 2.º semestre de 2026.

GARCIA, F.; MAGALHÃES, A. H. *Aula 4 – Linguagem de descrição de hardware (HDL): pacote IEEE-1164 e código concorrente*. 2023. Slides da disciplina Sistemas Reconfiguráveis. Rev. G.

GARCIA, F.; MAGALHÃES, A. H. *Aula 3 – Linguagem de descrição de hardware (HDL): unidades fundamentais, classes, tipos e operadores*. 2024. Slides da disciplina Sistemas Reconfiguráveis. Rev. D.

# Apêndices

# APÊNDICE A – CÓDIGO VHDL COMPLETO DO REG_BANK

Código A.1 – Arquivo `Trabalho2_RegBank.vhd`

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY Trabalho2_RegBank IS
6      PORT(
7          a_do   : OUT STD_LOGIC_VECTOR(7 downto 0); -- Saída de dado de um dos registradores selecionado pela
               entrada a_sel
8          b_do   : OUT STD_LOGIC_VECTOR(7 downto 0); -- Saída de dado de um dos registradores selecionado pela
               entrada b_sel
9          nrst   : IN STD_LOGIC; -- Entrada de reset assíncrono
10         clk_in : IN STD_LOGIC; -- Entrada de clock para escrita em todos os registradores
11         di     : IN STD_LOGIC_VECTOR(7 downto 0) := (others => '0'); -- Entrada de dados para escrita nos
               registradores R0 a R3
12         a_sel  : IN STD_LOGIC_VECTOR(1 downto 0) := "00"; -- Entrada para selecionar qual registrador vai ser lido
                na saída a_do
13         -- e será escrito quanto a operação de escrita estiver habilitada por wr_ena.
14         b_sel  : IN STD_LOGIC_VECTOR(1 downto 0) := "00"; -- Entrada para selecionar qual registrador vai ser lido
                na saída b_do
15         wr_ena : IN STD_LOGIC := '0' -- Entrada de habilitação para escrita nos registradores
16     );
17 END ENTITY Trabalho2_RegBank;
18
19 ARCHITECTURE Reg_Bank OF Trabalho2_RegBank IS
20
21     SIGNAL R0, R1, R2, R3 : STD_LOGIC_VECTOR (7 DOWNTO 0);
22
23 BEGIN
24
25     registrador : PROCESS(nrst, clk_in)
26     BEGIN
27         IF nrst = '0' THEN
28             R0 <= "00000000";
29             R1 <= "00000000";
30             R2 <= "00000000";
31             R3 <= "00000000";
32         ELSIF RISING_EDGE(clk_in) THEN
33             IF wr_ena = '1' THEN
34                 CASE a_sel IS
35                     WHEN "00" => R0 <= di;
36                     WHEN "01" => R1 <= di;
37                     WHEN "10" => R2 <= di;
38                     WHEN "11" => R3 <= di;
39                 END CASE;
40             END IF;
41         END IF;
42     END PROCESS registrador;
43
44     WITH a_sel SELECT
45         a_do <= R0 WHEN "00",
46                 R1 WHEN "01",
47                 R2 WHEN "10",
48                 R3 WHEN "11";
49
50     WITH b_sel SELECT
51         b_do <= R0 WHEN "00",
52                 R1 WHEN "01",
53                 R2 WHEN "10",
54                 R3 WHEN "11";
55
56 END ARCHITECTURE Reg_Bank;
```

# APÊNDICE B – CÓDIGO VHDL COMPLETO DO STATUS_REG

Código B.1 – Arquivo `Trabalho2_RegStatus.vhd`

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY Trabalho2_RegStatus IS
6      PORT(
7          nrst : IN STD_LOGIC; -- Entrada de reset assíncrono
8          clk_in : IN STD_LOGIC; -- Entrada de clock para escrita no registrador
9          status_reg_in : IN STD_LOGIC_VECTOR (4 DOWNTO 0); -- Entrada de dado para escrita nos registradores [0]=RN
                 [1]=RV [2]=RZ [3]=RH [4]=RC
10         status_reg_wr_ena : IN STD_LOGIC_VECTOR (4 DOWNTO 0); -- Entrada de habilitação para escrita registradores
11         status_reg_out : OUT STD_LOGIC_VECTOR (4 DOWNTO 0) -- Saída dos registradores RN, RV, RZ, RH e RC
12     );
13 END ENTITY Trabalho2_RegStatus;
14
15 ARCHITECTURE Reg_Status OF Trabalho2_RegStatus IS
16     SIGNAL registradores: STD_LOGIC_VECTOR (4 DOWNTO 0); -- Vetor que faz o papel dos registradores, [0]=RN [1]=RV
           [2]=RZ [3]=RH [4]=RC
17 BEGIN
18     Reg_Status : PROCESS(nrst, clk_in)
19     BEGIN
20         IF nrst = '0' THEN
21             registradores <= (OTHERS => '0');
22         ELSIF RISING_EDGE (clk_in) THEN
23             FOR i IN 0 to 4 LOOP
24                 IF status_reg_wr_ena(i) = '1' THEN
25                     registradores(i) <= status_reg_in(i);
26                 END IF;
27             END LOOP;
28         END IF;
29     END PROCESS Reg_Status;
30
31     status_reg_out <= registradores;
32
33 END ARCHITECTURE Reg_Status;
```

# APÊNDICE C – CÓDIGO VHDL COMPLETO DA STACK

Código C.1 – Arquivo `Trabalho2_Stack.vhd`

```
1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY Trabalho2_Stack IS
6      PORT(
7          nrst       : IN STD_LOGIC; -- Entrada de reset assincrono
8          clk_in     : IN STD_LOGIC; -- Entrada de clock para escrita nos registradores
9          stack_in   : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); -- Entrada para a pilha
10         stack_push : IN STD_LOGIC := '0'; -- Entrada de habilitação para colocar valores na pilha (tem preferência
               sobre o pop)
11         stack_pop  : IN STD_LOGIC := '0'; -- Entrada de habilitação para retirar valores da pilha
12         stack_out  : OUT STD_LOGIC_VECTOR(10 DOWNTO 0) -- Saída correspondente à primeira posição (topo) da pilha
13     );
14 END ENTITY Trabalho2_Stack;
15
16 ARCHITECTURE Stack OF Trabalho2_Stack IS
17
18     -- Tipo que representa os 8 registradores de 11 bits da pilha, pilha(0) é o topo
19     TYPE tipo_pilha IS ARRAY (0 TO 7) OF STD_LOGIC_VECTOR(10 DOWNTO 0);
20     SIGNAL pilha : tipo_pilha;
21
22 BEGIN
23
24     Stack : PROCESS(nrst, clk_in)
25     BEGIN
26         IF nrst = '0' THEN
27             pilha <= (OTHERS => (OTHERS => '0'));
28         ELSIF RISING_EDGE(clk_in) THEN
29             IF stack_push = '1' THEN
30                 -- push: o valor de entrada vai para o topo e cada registrador recebe o conteudo do anterior
31                 -- o conteudo do oitavo registrador e perdido (stack overflow)
32                 pilha(0) <= stack_in;
33                 FOR i IN 1 TO 7 LOOP
34                     pilha(i) <= pilha(i-1);
35                 END LOOP;
36             ELSIF stack_pop = '1' THEN
37                 -- pop: cada registrador recebe o conteudo do seguinte e o oitavo recebe zero
38                 FOR i IN 0 TO 6 LOOP
39                     pilha(i) <= pilha(i+1);
40                 END LOOP;
41                 pilha(7) <= (OTHERS => '0');
42             END IF;
43         END IF;
44     END PROCESS Stack;
45
46     stack_out <= pilha(0);
47
48 END ARCHITECTURE Stack;
```

# APÊNDICE D – CÓDIGO VHDL COMPLETO DO PROG_CNT

Código D.1 – Arquivo `Trabalho2_ProgCnt.vhd`

```vhdl
LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
USE IEEE.NUMERIC_STD.ALL;

ENTITY Trabalho2_ProgCnt IS
    PORT(
        nrst        : IN STD_LOGIC; -- Entrada de reset assíncrono
        clk_in      : IN STD_LOGIC; -- Entrada de clock para incremento ou carga do contador
        pc_ctrl     : IN STD_LOGIC_VECTOR(1 DOWNTO 0) := "00"; -- Controle: 00 mantém, 01 carrega from_stack, 10 carrega new_pc_in, 11 incrementa
        new_pc_in   : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); -- Entrada para carga no contador quando pc_ctrl = "10"
        from_stack  : IN STD_LOGIC_VECTOR(10 DOWNTO 0) := (OTHERS => '0'); -- Entrada para carga no contador quando pc_ctrl = "01"
        next_pc_out : OUT STD_LOGIC_VECTOR(10 DOWNTO 0); -- Saída do próximo valor do contador (combinacional)
        pc_out      : OUT STD_LOGIC_VECTOR(10 DOWNTO 0) -- Saída do contador
    );
END ENTITY Trabalho2_ProgCnt;

ARCHITECTURE Prog_Cnt OF Trabalho2_ProgCnt IS

    SIGNAL pc      : STD_LOGIC_VECTOR(10 DOWNTO 0); -- Saída dos flip-flops do contador
    SIGNAL next_pc : STD_LOGIC_VECTOR(10 DOWNTO 0); -- Entrada dos flip-flops do contador

BEGIN

    -- Parte combinacional: calcula o valor a ser carregado na próxima borda de subida do clock
    -- como o vetor tem 11 bits, o incremento de 2047 volta naturalmente para 0 (módulo 2048)
    Combinacional : PROCESS(pc_ctrl, pc, new_pc_in, from_stack)
    BEGIN
        CASE pc_ctrl IS
            WHEN "01"   => next_pc <= from_stack;
            WHEN "10"   => next_pc <= new_pc_in;
            WHEN "11"   => next_pc <= STD_LOGIC_VECTOR(UNSIGNED(pc) + 1);
            WHEN OTHERS => next_pc <= pc;
        END CASE;
    END PROCESS Combinacional;

    -- Parte sequencial: registra o próximo valor na borda de subida do clock
    Sequencial : PROCESS(nrst, clk_in)
    BEGIN
        IF nrst = '0' THEN
            pc <= (OTHERS => '0');
        ELSIF RISING_EDGE(clk_in) THEN
            pc <= next_pc;
        END IF;
    END PROCESS Sequencial;

    next_pc_out <= next_pc;
    pc_out <= pc;

END ARCHITECTURE Prog_Cnt;
```